CSA4704-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 8: Two-Layer Neural Network with Backpropagation

Problem Statement: The bank's screener from Experiment 7 fails on non-linear cases. The data team needs a hidden layer and must understand how errors flow back. The XOR-style loan dataset and a two-
moons customer-segmentation dataset. The objective is to build a 2-layer network from scratch and implement backpropagation manually. Use NumPy, Matplotlib.

Tasks:

1. Initialise weights for an input–hidden–output network in NumPy.
2. Write the forward pass with sigmoid activations.
3. Derive and code the backward pass using the chain rule.
4. Train with gradient descent and plot loss against epochs.
5. Plot the non-linear decision boundary; verify gradients numerically for one weight.

FORMULA:

1. Weight Initialization

W_1 = Random weights between input and hidden layers

W_2 = Random weights between hidden and output layers

2. Weighted Sum

$z_1 = XW_1 + b_1$

$z_2 = H\,W_2 + b_2$

3. Sigmoid Activation Function

$\sigma(z) = 1 / (1 + e^{-z})$

4. Forward Propagation

$H = \sigma(XW_1 + b_1)$

$\hat{Y} = \sigma(HW_2 + b_2)$

5. Binary Cross-Entropy Loss Function

$L = -(1/N)\ \Sigma\ [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$

6. Backpropagation Error

$\delta_2 = \hat{Y} - y$

$\delta_1 = (\delta_2\ W_2^T) \odot H \odot (1 - H)$

7. Gradient Calculation

$\partial L/\partial W_2 = H^T\delta_2 / N$

$\partial L/\partial b_2 = \Sigma\delta_2 / N$

$\partial L/\partial W_1 = X^T\delta_1 / N$

$\partial L/\partial b_1 = \Sigma\delta_1 / N$

8. Gradient Descent Weight Update

$W_1(new) = W_1(old) - \eta(\partial L/\partial W_1)$

$W_2(new) = W_2(old) - \eta(\partial L/\partial W_2)$

$b_1(new) = b_1(old) - \eta(\partial L/\partial b_1)$

$b_2(new) = b_2(old) - \eta(\partial L/\partial b_2)$

9. Numerical Gradient Verification

$\text{Numerical Gradient} = [L(W + \varepsilon) - L(W - \varepsilon)] / (2\varepsilon)$

10. Accuracy

Accuracy = (Correct Predictions / Total Predictions) × 100%

CODE:

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# 1. Sigmoid activation
def sigmoid(z):
    z = np.clip(z, -500, 500)
    return 1 / (1 + np.exp(-z))

…    return dW1[0, 0]

# Use trained XOR network for gradient verification
analytic = analytical_gradient(X_xor, Y_xor, W1x, b1x, W2x, b2x)
numeric = numerical_gradient(X_xor, Y_xor, W1x, b1x, W2x, b2x)

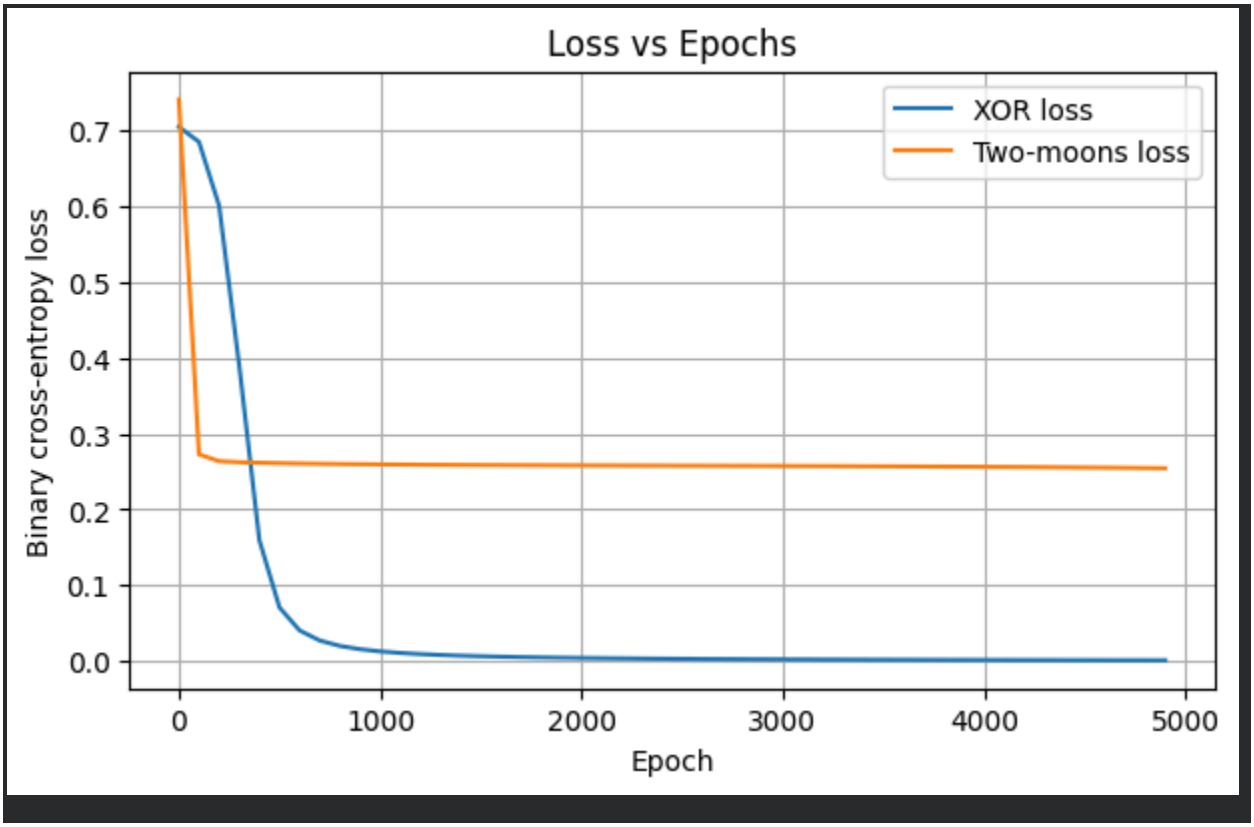
print("\nGRADIENT VERIFICATION")
print("Analytical gradient:", analytic)
print("Numerical gradient: ", numeric)
print("Absolute difference:", abs(analytic - numeric))
```

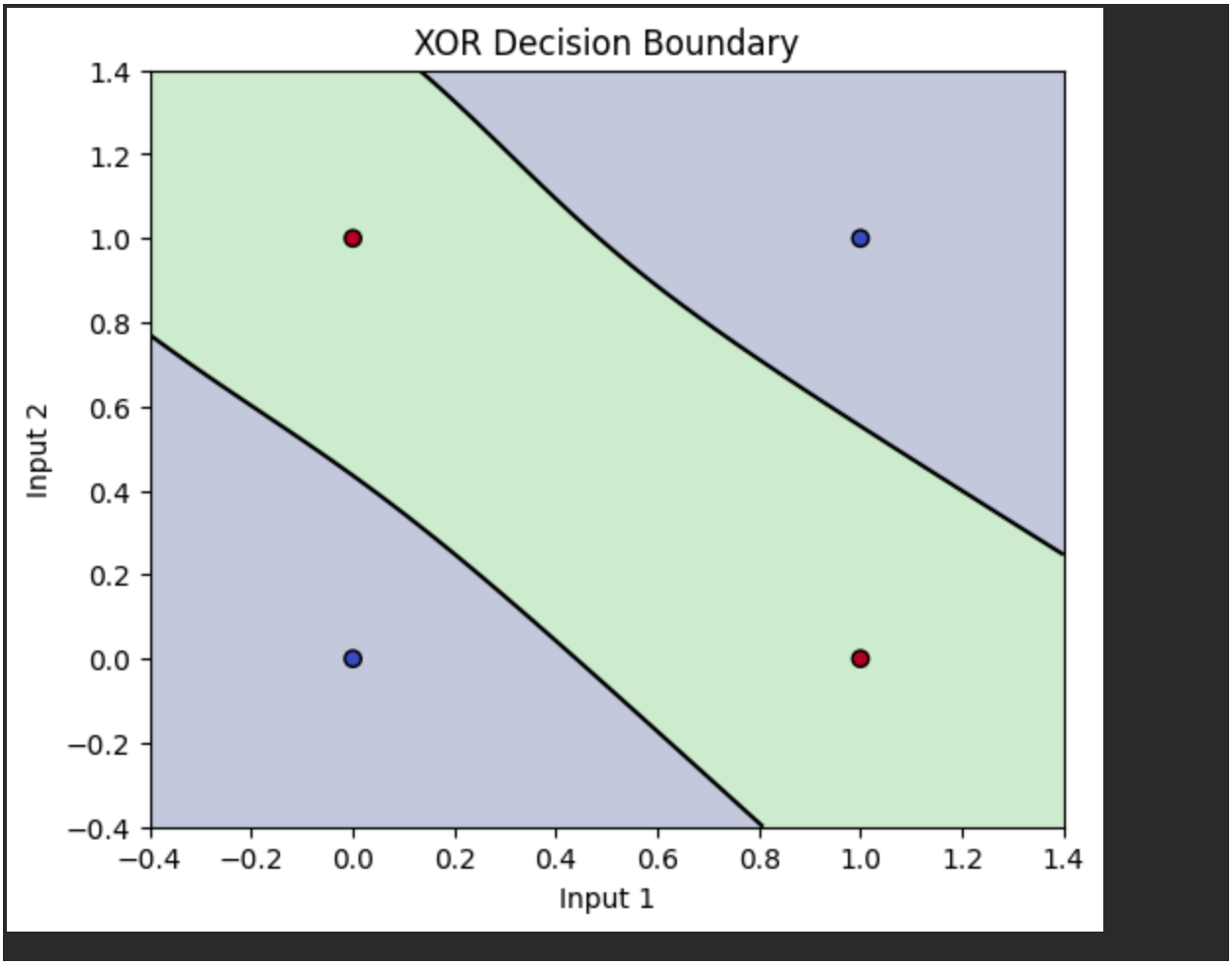
OUTPUT:

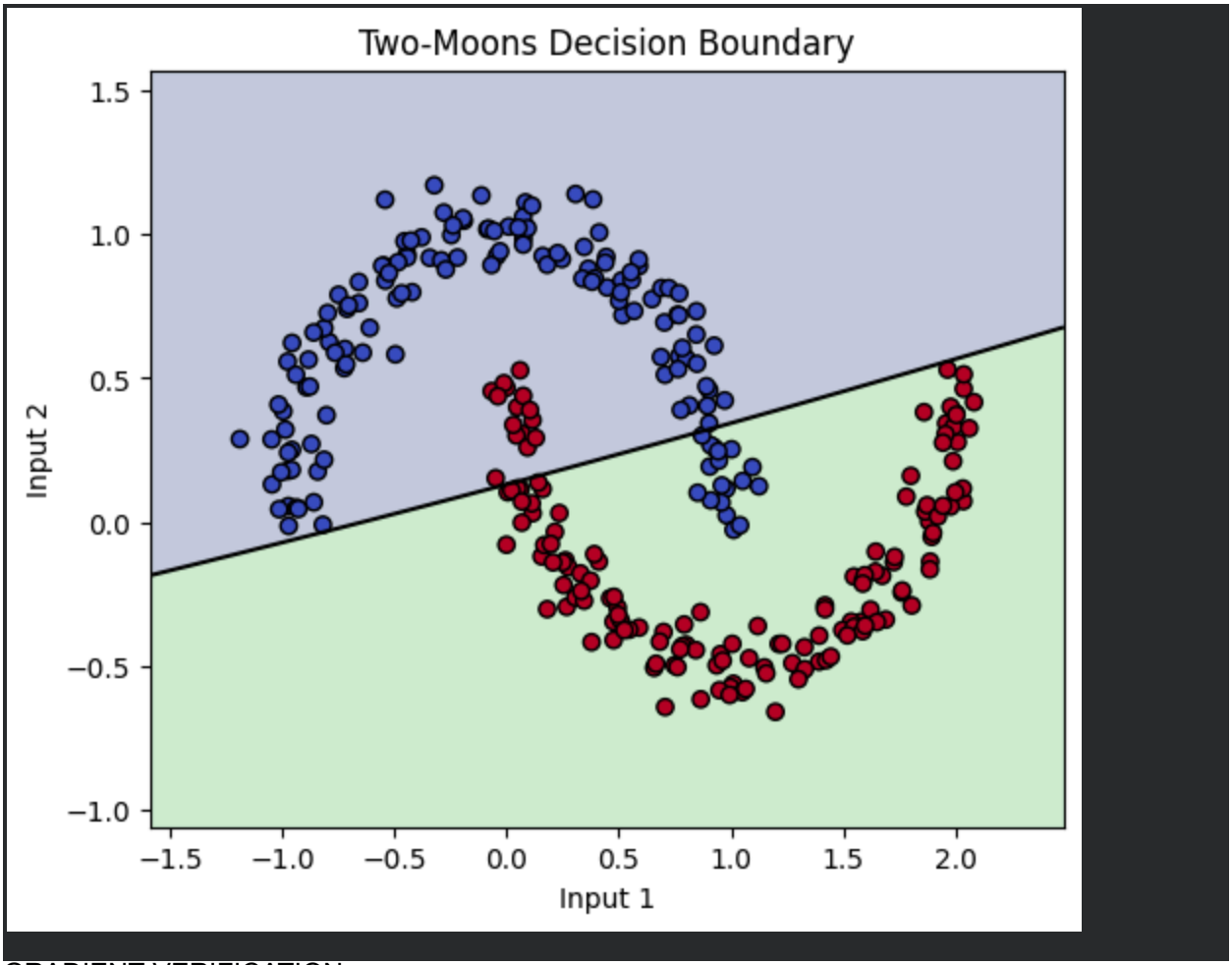
```
XOR predictions:
[[0.002]
 [0.999]
 [0.998]
 [0.001]]
XOR classes: [0 1 1 0]
```

XOR accuracy: 100.0 %

Two-moons accuracy: 89.0 %




XOR Decision Boundary
Input 2
Input 1
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.4
−0.2
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4



GRADIENT VERIFICATION

Analytical gradient: -9.223986805103095e-05

Numerical gradient: -9.223986870534119e-05

Absolute difference: 6.543102377184334e-13